

SCALE200

PRODUCTION READINESS

KIT

From vibe-coded prototype to safer, more reliable launch

Security Checklist	28 checks
Deployment Checklist	22 steps
AI Testing Prompts	15 prompts
Privacy Requirements	18 checks
MVP Health Audit	25 checks

128 practical checks, steps and prompts

Version 1.1 | August 2026

How to use this kit

Work through the kit before a public launch, after a major architectural change, and before handling sensitive customer data. Assign an owner and due date to every failed or uncertain item. Keep evidence such as screenshots, test logs, links to configuration, pull requests and incident runbooks.

PASS	REVIEW	FAIL	N/A
☐	☐	☐	☐

Recommended workflow

1. Complete the MVP Health Audit for a fast baseline.
2. Run the Security and Privacy checklists.
3. Complete Deployment checks in a staging environment.
4. Use the AI prompts to expand test coverage, then verify results manually.
5. Complete the final Launch Checklist in production.
6. Create a prioritized launch action plan: Blocker, Before launch, After launch.

Important

This kit is an educational self-assessment, not a penetration test, legal opinion, compliance certification or guarantee of security. AI-generated test ideas can be incomplete or incorrect; a qualified person must review and execute them safely. Seek security, privacy and legal professionals for high-risk systems or regulated data.

1. Security Checklist

A practical launch review for authentication, access control, secrets, APIs, data protection and operations.

28 ITEMS

Use: Pass / Review / Fail / N/A

1. Passwords are securely hashed

Why it matters: Plain-text or reversible passwords turn one database leak into an account takeover event.

How to verify: Inspect the user table and authentication code. Confirm a modern adaptive hash such as Argon2id, bcrypt or PBKDF2 is used and unique salts are applied.

Action if missing: Replace custom password storage with a maintained authentication library and force password resets after migration.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

2. Login attempts are rate-limited

Why it matters: Automated attacks can guess weak or reused passwords at high speed.

How to verify: Try repeated failed logins from one account and one IP. Confirm throttling, temporary lockout or challenge controls activate without enabling easy denial-of-service.

Action if missing: Add layered rate limits by IP, account and device signals, plus alerting for unusual failures.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

3. Multi-factor authentication protects privileged users

Why it matters: A stolen password should not be enough to take over an administrator or billing account.

How to verify: Check whether administrators and high-risk roles can or must enroll in MFA, and test recovery paths.

Action if missing: Require phishing-resistant MFA where practical and secure the recovery process.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

4. Password reset is safe

Why it matters: Weak reset flows can bypass an otherwise secure login.

How to verify: Confirm reset tokens are random, single-use, short-lived, stored securely and do not reveal whether an email exists.

Action if missing: Use the identity provider's supported reset flow; invalidate sessions after a successful reset.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

5. Email or identity ownership is verified

Why it matters: Unverified identities can be used for fraud, spam or account confusion.

How to verify: Create an account with an email you do not control and confirm sensitive features remain blocked until verification.

Action if missing: Gate risky actions behind verified ownership and expire unused verification links.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

6. Sessions expire and can be revoked

Why it matters: Long-lived or non-revocable tokens increase the impact of theft.

How to verify: Test idle timeout, absolute timeout, logout, password change and administrator revocation.

Action if missing: Use short-lived access tokens, controlled refresh tokens and server-side revocation where needed.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

7. Authentication cookies are hardened

Why it matters: Browser cookies can be exposed through insecure transport or client-side scripts.

How to verify: Inspect production cookies for Secure, HttpOnly and appropriate SameSite attributes, domain and path scope.

Action if missing: Configure cookie flags centrally and avoid storing sensitive tokens in localStorage when secure cookies are suitable.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

8. Every protected action checks permission server-side

Why it matters: Hidden buttons do not stop a user from calling the underlying API directly.

How to verify: Call protected endpoints with a lower-privileged account and by modifying client requests.

Action if missing: Add authorization middleware or policy checks at every protected server-side action.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

9. Users cannot access another user's records

Why it matters: Changing an object ID is a common way to expose another customer's data.

How to verify: Change IDs in URLs, API bodies and GraphQL variables. Test read, update and delete operations.

Action if missing: Check object ownership or tenant membership on every object access.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

10. Roles and permissions are explicit

Why it matters: Vague role logic often creates accidental administrator access.

How to verify: List every role and permitted action. Test each role against the matrix.

Action if missing: Create a documented permission model and deny access by default.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

11. Administration surfaces are separately protected

Why it matters: An obscure admin URL is not a security control.

How to verify: Attempt direct access to admin pages and APIs with anonymous and normal user accounts.

Action if missing: Require explicit privileged authorization, stronger authentication and audit logging.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

12. Least privilege is applied

Why it matters: Overpowered users, services and keys make every mistake more damaging.

How to verify: Review cloud roles, database grants, API scopes and service accounts.

Action if missing: Remove unused permissions and create separate identities for separate services.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

13. No secrets exist in source code or frontend bundles

Why it matters: Anything committed or shipped to the browser should be considered exposed.

How to verify: Search current code and Git history for keys, tokens, passwords, private URLs and credentials; inspect browser bundles.

Action if missing: Revoke exposed secrets, clean history where appropriate and move values to protected configuration.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

14. Production secrets use managed storage

Why it matters: Loose environment files and shared messages are easily copied or leaked.

How to verify: Review how production credentials are injected, accessed, backed up and audited.

Action if missing: Use the hosting platform's secret store or a dedicated secret manager with access control.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

15. Development, test and production are separated

Why it matters: Shared data and credentials allow testing mistakes to affect real customers.

How to verify: Compare databases, keys, domains, storage and accounts across environments.

Action if missing: Create isolated environments and prevent test systems from using production credentials.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

16. Credentials can be rotated quickly

Why it matters: A leaked key remains dangerous until it is replaced everywhere.

How to verify: Choose one non-critical key and document the full rotation procedure, including rollback.

Action if missing: Add ownership, expiry where supported and an emergency rotation runbook.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

17. Debug and developer modes are disabled in production

Why it matters: Stack traces and debug tools expose internal details attackers can use.

How to verify: Trigger errors in production and inspect responses, headers and accessible development endpoints.

Action if missing: Use generic user errors, protected structured logs and production configuration checks.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

18. Private APIs require authentication

Why it matters: The backend must not trust that requests came through the intended interface.

How to verify: Call each non-public endpoint without credentials and with malformed or expired credentials.

Action if missing: Enforce authentication centrally and maintain an explicit public-endpoint list.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

19. API authorization is checked per object and property

Why it matters: Authenticated users may still try to access forbidden records or fields.

How to verify: Test object IDs, nested resources, bulk actions and fields not shown in the UI.

Action if missing: Apply object-level, function-level and property-level authorization.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

20. Sensitive and costly endpoints are rate-limited

Why it matters: Unrestricted requests can enable spam, scraping, AI cost abuse and service outages.

How to verify: Stress login, signup, email, search, upload, export, AI and payment-adjacent flows within safe test limits.

Action if missing: Set endpoint-specific quotas, concurrency controls, payload limits and monitoring.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

21. All external input is validated server-side

Why it matters: Client-side validation can be bypassed and AI-generated input may be unexpected.

How to verify: Send missing, extra, oversized, wrong-type and boundary values directly to APIs.

Action if missing: Use allow-list schemas and reject unknown or malformed fields.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

22. Injection risks are controlled

Why it matters: Untrusted input can alter database queries, commands, templates or rendered pages.

How to verify: Review query construction and test representative SQL, command, template and script payloads in a safe environment.

Action if missing: Use parameterized queries, safe APIs, output encoding and avoid shell construction.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

23. API responses expose only required fields

Why it matters: Returning full internal objects can leak sensitive data and implementation details.

How to verify: Inspect successful and error responses for hidden fields, secrets, internal metadata and personal data.

Action if missing: Use explicit response schemas and field allow-lists.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

24. CORS is restricted

Why it matters: Overly broad browser access can allow untrusted websites to interact with authenticated APIs.

How to verify: Inspect Access-Control-Allow-Origin, credentials, methods and headers for production APIs.

Action if missing: Allow only approved origins and avoid wildcard origins with credentialed requests.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

25. HTTPS is enforced everywhere

Why it matters: Unencrypted traffic allows interception and modification.

How to verify: Check every domain and subdomain, HTTP redirects, certificate validity and mixed content.

Action if missing: Redirect HTTP to HTTPS, automate renewal and enable secure transport settings.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

26. Sensitive data is minimized and protected

Why it matters: Collecting or logging unnecessary data increases privacy and breach impact.

How to verify: Trace sensitive values through forms, URLs, databases, analytics, logs, backups and exports.

Action if missing: Remove unnecessary fields, encrypt where appropriate and redact logs.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

27. File uploads are constrained and isolated

Why it matters: Uploads can hide malware, oversized payloads or executable content.

How to verify: Test file type, actual content, size, filename, path handling, download headers and storage location.

Action if missing: Allow-list types and sizes, rename files, scan where appropriate and store outside executable paths.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

28. Security events, dependencies and backups are monitored

Why it matters: Teams need to detect attacks, patch known issues and recover from failures.

How to verify: Review alerts, audit logs, dependency scanning, update ownership, backup frequency and a recent restore test.

Action if missing: Create a minimum monitoring dashboard, patch routine, incident contacts and tested recovery procedure.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

Dimension	Passed	Total	Score
Authentication & access	___	12	___ %
Secrets & configuration	___	5	___ %
API & backend	___	7	___ %
Data & operations	___	4	___ %
TOTAL	___	28	___ %

24-28 Strong foundation: Proceed only after reviewing remaining gaps.

18-23 Needs improvement: Resolve material issues before a public launch.

10-17 High risk: Do not handle sensitive or valuable data until major gaps are fixed.

0-9 Not production-ready: Pause launch and obtain professional help.

2. Deployment Checklist

A repeatable path from repository to monitored production with recovery options.

22 ITEMS

Use: Pass / Review / Fail / N/A

1. Define the production architecture

Why it matters: A simple diagram prevents hidden dependencies and unclear ownership.

How to verify: Draw users, frontend, backend, database, storage, third parties, DNS and monitoring. Mark trust boundaries.

Action if missing: Create a one-page architecture diagram and keep it with the repository.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

2. Create isolated production resources

Why it matters: Production should not share mutable infrastructure or data with development.

How to verify: Verify separate accounts/projects, database, storage, keys and configuration.

Action if missing: Provision dedicated production resources using repeatable scripts or platform configuration.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

3. Pin runtime and dependency versions

Why it matters: Unpinned versions make builds unpredictable.

How to verify: Check runtime files, lockfiles, container tags and package manager configuration.

Action if missing: Pin versions and define an intentional update process.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

4. Produce a reproducible build

Why it matters: The same commit should create the same deployable output.

How to verify: Build twice in a clean environment and compare outputs or behavior.

Action if missing: Move undocumented local steps into scripts and CI.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

5. Containerize or standardize the runtime

Why it matters: A defined runtime reduces “works on my machine” failures.

How to verify: Start the app from a clean host using only documented files and configuration.

Action if missing: Add a production Dockerfile or equivalent platform build definition.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

6. Run as a non-root, minimal service

Why it matters: Excess privileges increase the impact of compromise.

How to verify: Inspect the runtime user, image contents and filesystem permissions.

Action if missing: Use a minimal base image and a non-root user; remove build tools from runtime.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

7. Separate configuration from code

Why it matters: Environment differences should not require editing source files.

How to verify: List all environment-specific values and confirm they are injected at runtime or build time appropriately.

Action if missing: Create validated configuration with safe defaults and clear required variables.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

8. Store secrets outside the repository

Why it matters: Deployment should obtain secrets through controlled channels.

How to verify: Review CI variables, platform settings and access logs.

Action if missing: Use protected secrets and environment-scoped access.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

9. Validate required environment variables

Why it matters: Missing configuration should fail early, not during customer use.

How to verify: Start with required values removed or malformed and confirm startup fails clearly.

Action if missing: Add a typed configuration schema and startup validation.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

10. Automate tests in CI

Why it matters: Every change should receive the same minimum quality checks.

How to verify: Confirm pull requests run linting, unit tests, build and core integration checks.

Action if missing: Create a blocking CI workflow with clear failure output.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

11. Add security checks to CI

Why it matters: Known vulnerable dependencies and leaked secrets should be caught before release.

How to verify: Review dependency, secret and static analysis steps and their failure thresholds.

Action if missing: Enable maintained scanners and define how findings are triaged.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

12. Use a controlled deployment workflow

Why it matters: Manual ad-hoc server changes are hard to review and reproduce.

How to verify: Trace how a commit reaches production and who can approve or trigger it.

Action if missing: Deploy from CI/CD using protected branches/environments and auditable permissions.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

13. Run database migrations safely

Why it matters: Schema changes can lock tables, lose data or break older app versions.

How to verify: Test migrations against a production-like copy and test rollback or forward-fix.

Action if missing: Use versioned migrations, backups and compatible rollout patterns.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

14. Configure domain and DNS deliberately

Why it matters: Incorrect DNS causes downtime and can leave abandoned records.

How to verify: Verify A/AAAA/CNAME records, ownership, TTLs and obsolete records.

Action if missing: Document DNS ownership and remove unused records.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

15. Enable and renew TLS certificates

Why it matters: Expired or incomplete certificates block users and weaken trust.

How to verify: Test all hostnames, chain validity, renewal automation and alerts.

Action if missing: Automate certificates and monitor expiry.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

16. Add application health checks

Why it matters: The platform needs a reliable signal that the app is alive and ready.

How to verify: Test liveness and readiness endpoints during startup, database failure and dependency failure.

Action if missing: Create lightweight health endpoints that avoid exposing sensitive details.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

17. Centralize logs and errors

Why it matters: Local logs disappear during crashes and scaling.

How to verify: Find one recent error from request to stack trace and verify correlation data.

Action if missing: Send structured logs and exceptions to a controlled monitoring system.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

18. Monitor availability and key flows

Why it matters: A green server does not mean checkout, signup or login works.

How to verify: Check uptime probes and synthetic tests for the most important user journey.

Action if missing: Add external uptime monitoring and at least one critical-flow check.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

19. Set resource limits and scaling rules

Why it matters: Unbounded memory, CPU or concurrency can cause cascading failure or surprise cost.

How to verify: Inspect limits, timeouts, worker counts, autoscaling and queue behavior.

Action if missing: Define conservative limits and load-test before increasing them.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

20. Back up stateful data and test restore

Why it matters: A backup is only useful if it can be restored.

How to verify: Confirm schedule, retention, encryption, access and the date of the last successful restore test.

Action if missing: Perform a documented restore exercise and assign an owner.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

21. Prepare rollback and incident steps

Why it matters: Teams need a safe response when a release fails.

How to verify: Simulate a bad release and confirm rollback, communication and ownership are clear.

Action if missing: Create a short release rollback and incident runbook.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

22. Run a final production smoke test

Why it matters: Configuration errors often appear only after deployment.

How to verify: After release, test homepage, signup/login, core transaction, email, upload, payment sandbox where applicable and monitoring.

Action if missing: Use a repeatable smoke checklist and record evidence for every release.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

Dimension	Passed	Total	Score
Build & runtime	___	6	___ %
Configuration & CI	___	6	___ %
Release & infrastructure	___	6	___ %
Operations & recovery	___	4	___ %
TOTAL	___	22	___ %

20-22 Ready with minor follow-up: Record remaining risks and owners.

15-19 Conditional readiness: Fix launch-critical gaps.

8-14 High operational risk: Deployment is likely to be fragile.

0-7 Not ready: Establish a repeatable production process before launch.

3. AI Testing Prompts

Prompt templates for generating test ideas. Review every output and run tests only in authorized environments.

15 ITEMS	Use: Pass / Review / Fail / N/A
----------	---------------------------------

1. Requirements and user-journey review

Use this to: Find missing journeys and ambiguous behavior before testing details

You are a senior QA engineer reviewing an AI-built web application. Based on the following product description, list the critical user journeys, required preconditions, expected outcomes, failure paths and data that must be prepared. Identify ambiguities as questions rather than inventing behavior. Product description: [PASTE]. User roles: [PASTE].

Evidence to keep: Approved journey list and unresolved questions

☐ Run Result: ☐ Pass ☐ Issues found ☐ Retest needed

2. Happy-path test cases

Use this to: Create a clean baseline test set

Create concise step-by-step happy-path test cases for this feature: [FEATURE]. For each case provide role, preconditions, test data, steps, expected result and evidence to capture. Do not assume access beyond the stated role.

Evidence to keep: Executed cases with screenshots or logs

☐ Run Result: ☐ Pass ☐ Issues found ☐ Retest needed

3. Boundary-value tests

Use this to: Catch off-by-one, length and numeric errors

For the following fields and rules, generate boundary-value tests covering minimum, just below minimum, maximum, just above maximum, empty, null, zero, negative, decimal, very large and unusual-but-valid values where applicable: [FIELDS/RULES]. Return a table with input and expected behavior.

Evidence to keep: Results for every relevant boundary

☐ Run Result: ☐ Pass ☐ Issues found ☐ Retest needed

4. Invalid and malformed input

Use this to: Test server-side validation

Act as a defensive application tester. For endpoint/form [NAME], create safe malformed-input tests for missing fields, extra fields, wrong data types, invalid formats, duplicate values, oversized payloads and unsupported characters. Do not include destructive exploit instructions. State expected status and user-facing error behavior.

Evidence to keep: Request/response samples and validation defects

☐ Run Result: ☐ Pass ☐ Issues found ☐ Retest needed

5. Authentication flow review

Use this to: Test signup, login and recovery

Create a test matrix for signup, email verification, login, logout, session expiry, password reset and account recovery for this app: [DETAILS]. Include normal, failed, repeated, expired-token and multi-device scenarios. Highlight account-enumeration risks.

Evidence to keep: Authentication matrix and defects

☐ Run Result: ☐ Pass ☐ Issues found ☐ Retest needed

6. Authorization and tenant isolation

Use this to: Find cross-user access problems

Given these roles and resources [PASTE], generate authorization tests for read, create, update, delete, export and bulk actions. Include attempts to change object IDs, tenant IDs and hidden fields. Expected behavior must be deny-by-default and should not leak whether forbidden objects exist.

Evidence to keep: Role/resource permission evidence

☐ Run Result: ☐ Pass ☐ Issues found ☐ Retest needed

7. Concurrency and duplicate actions

Use this to: Find race conditions and double submissions

Generate concurrency tests for [FEATURE], including double-clicks, repeated requests, two browser tabs, two devices, retry after timeout and simultaneous updates by different users. Explain the correct idempotent or conflict behavior for each case.

Evidence to keep: Duplicate/race test results

☐ Run Result: ☐ Pass ☐ Issues found ☐ Retest needed

8. Network and dependency failure

Use this to: Test resilience to partial outages

Create failure-mode tests for [FEATURE] when the network is slow, offline or interrupted and when each dependency times out, returns an error, returns malformed data or succeeds after a retry. Include expected user messaging, retry behavior and data consistency.

Evidence to keep: Failure simulation results and recovery behavior

☐ Run Result: ☐ Pass ☐ Issues found ☐ Retest needed

9. Database and persistence behavior

Use this to: Catch lost or inconsistent data

For workflow [WORKFLOW], identify all state transitions and create persistence tests for refresh, logout/login, server restart, retry, partial save, rollback and concurrent modification. State which data must be atomic and which operations must be idempotent.

Evidence to keep: State-transition test record

☐ Run Result: ☐ Pass ☐ Issues found ☐ Retest needed

10. File upload testing

Use this to: Check file handling safely

Create a safe file-upload test plan for [UPLOAD FEATURE]. Cover allowed/disallowed extensions, MIME mismatch, empty files, maximum size, multiple files, duplicate names, Unicode names, interrupted upload, preview/download and unauthorized access. Do not generate malware.

Evidence to keep: Upload matrix and storage/access findings

☐ Run Result: ☐ Pass ☐ Issues found ☐ Retest needed

11. Email and notification testing

Use this to: Verify communications without spamming

Create test cases for [EMAIL/NOTIFICATION FLOW] covering correct recipient, content variables, localization, duplicate prevention, unsubscribe or preferences, bounce/error handling, delayed delivery and sensitive-data exposure. Use sandbox recipients only.

Evidence to keep: Captured messages and delivery logs

☐ Run Result: ☐ Pass ☐ Issues found ☐ Retest needed

12. Performance and load scenarios

Use this to: Identify bottlenecks before launch

Design a staged performance test for [APP/FLOW] with expected daily users [N], peak concurrent users [N] and target response time [N]. Include baseline, normal load, peak, spike and soak tests; define metrics, pass criteria and safe stop conditions.

Evidence to keep: Load report with latency, errors and saturation

☐ Run Result: ☐ Pass ☐ Issues found ☐ Retest needed

13. Accessibility review

Use this to: Find common usability barriers

Review the described UI [PASTE OR SCREEN DETAILS] and create accessibility tests for keyboard-only use, focus order, labels, headings, error identification, contrast, zoom, screen-reader names and reduced motion. Reference observable behavior, not visual preference.

Evidence to keep: Accessibility test notes and priority fixes

☐ Run Result: ☐ Pass ☐ Issues found ☐ Retest needed

14. Privacy and data lifecycle

Use this to: Test collection, export and deletion

Create end-to-end privacy tests for the personal data used by this app: [DATA]. Cover consent or legal basis recording, notice display, data access/export, correction, deletion, retention expiry, backups, analytics and third-party transfers. Flag legal questions for professional review.

Evidence to keep: Data lifecycle evidence and legal questions

☐ Run Result: ☐ Pass ☐ Issues found ☐ Retest needed

15. Exploratory release challenge

Use this to: Use AI as a skeptical final reviewer

Act as a skeptical release reviewer. Here is the feature set, architecture and known limitations: [PASTE]. Propose the 20 highest-value exploratory tests most likely to uncover a launch-blocking issue. Rank by risk and explain what evidence would constitute pass or fail. Avoid repeating already-covered tests: [PASTE COVERAGE].

Evidence to keep: Ranked exploratory run and launch blockers

☐ Run Result: ☐ Pass ☐ Issues found ☐ Retest needed

AI testing rules

- Never paste production secrets, private customer data or confidential source code into an unapproved AI service.
- Treat generated cases as suggestions, not proof that the app works.
- Run potentially destructive tests only in an isolated environment with backups.
- Record which model, prompt, app version and environment produced each result.
- A human tester must confirm expected behavior and prioritize defects.

4. Privacy Requirements

A GDPR- and CCPA-aware self-check for small apps. Applicability and exact obligations require legal review.

18 ITEMS

Use: Pass / Review / Fail / N/A

1. Create a personal-data inventory

Why it matters: You cannot protect or disclose data you have not identified.

How to verify: List every personal data field, where it enters, why it is used, where it is stored, who can access it, and where it is sent.

Action if missing: Build a simple processing inventory and keep it updated with product changes.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

2. Define a lawful basis for each purpose

Why it matters: Under GDPR, processing needs a valid legal basis; consent is not automatically the right answer.

How to verify: For each processing purpose, record the proposed lawful basis and supporting rationale; obtain legal review for uncertain cases.

Action if missing: Stop undefined processing and document the basis before launch.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

3. Collect only necessary data

Why it matters: Extra data creates extra risk and compliance obligations.

How to verify: Challenge every field: is it required for the stated purpose, optional, or unnecessary?

Action if missing: Remove unnecessary collection and make optional fields genuinely optional.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

4. State specific purposes

Why it matters: Data should not be collected “just in case.”

How to verify: Compare forms, database fields and analytics events with documented purposes.

Action if missing: Write specific purposes and prevent incompatible reuse without review.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

5. Provide a clear privacy notice

Why it matters: People should understand who collects data, why, for how long and with whom it is shared.

How to verify: Review the notice against actual data flows and ensure it is accessible at collection points.

Action if missing: Draft plain-language layered notices and keep legal details accurate.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

6. Manage consent correctly where used

Why it matters: Consent must be meaningful, specific and withdrawable when it is the chosen basis.

How to verify: Confirm choices are not preselected, refusal is possible, records are kept and withdrawal is as easy as giving consent.

Action if missing: Implement granular preference records and withdrawal flows.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

7. Handle cookies and trackers

Why it matters: Non-essential analytics or advertising technologies may require prior choice depending on jurisdiction.

How to verify: Inventory cookies, SDKs, pixels and local storage; test behavior before and after user choices.

Action if missing: Block non-essential trackers until the required choice and remember preferences.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

8. Support access and data export

Why it matters: Users may have rights to receive information and copies of their personal data.

How to verify: Run a test request and verify identity checking, search scope, format, deadlines and third-party data protection.

Action if missing: Create a documented request workflow and export process.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

9. Support correction

Why it matters: Inaccurate data can harm users and violate data-quality duties.

How to verify: Test correction in profile, support and connected systems.

Action if missing: Provide correction routes and propagate updates to relevant processors.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

10. Support deletion and account closure

Why it matters: Deleting the visible account may not delete associated data.

How to verify: Trace deletion through databases, files, search indexes, analytics, vendors and backups.

Action if missing: Define deletion, anonymization and justified retention behavior.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

11. Define retention periods

Why it matters: Keeping data forever increases risk and may conflict with storage-limitation principles.

How to verify: Record retention by data category and purpose, including logs and backups.

Action if missing: Automate deletion or review dates and document exceptions.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

12. Protect children and sensitive data

Why it matters: Certain data and audiences can require stronger safeguards and legal analysis.

How to verify: Identify special-category data, precise location, biometrics, health, children or other high-risk processing.

Action if missing: Avoid unnecessary high-risk data and obtain specialist legal review.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

13. Sign appropriate processor agreements

Why it matters: Vendors handling personal data need clear contractual responsibilities.

How to verify: List processors and confirm data-processing terms, security commitments and sub-processors are reviewed.

Action if missing: Complete required agreements before sending production data.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

14. Review international transfers

Why it matters: Cloud and SaaS vendors may process data outside the user's region.

How to verify: Map hosting, support, analytics and sub-processors by country and transfer mechanism.

Action if missing: Choose suitable regions and obtain legal review of transfer safeguards.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

15. Secure personal data by design

Why it matters: Privacy depends on technical and organizational controls, not only policies.

How to verify: Review access, encryption, logging, testing, incident handling and least privilege for personal data.

Action if missing: Add safeguards proportional to sensitivity and risk.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

16. Prepare for personal-data breaches

Why it matters: Teams need to detect, assess, contain and document incidents quickly.

How to verify: Run a tabletop exercise covering discovery, evidence, containment, risk assessment, notification decisions and communications.

Action if missing: Create an incident plan with roles, contacts and decision records.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

17. Assess high-risk processing

Why it matters: Some processing may require a formal impact assessment before launch.

How to verify: Screen for systematic monitoring, large-scale sensitive data, profiling, novel technology or significant effects on people.

Action if missing: Seek privacy/legal advice and complete a DPIA or equivalent when required.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

18. Map US/California consumer-rights obligations

Why it matters: CCPA/CPRA applicability and obligations depend on business and data practices.

How to verify: Review whether the business falls within scope and whether it sells or shares personal

information; verify notice and request processes.

Action if missing: Obtain qualified legal advice and implement required rights, notices and opt-out mechanisms where applicable.

☐ Pass ☐ Review ☐ Fail ☐ N/A Owner: _____ Due: _____

Dimension	Passed	Total	Score
Data mapping & purpose	_____	5	_____ %
Transparency & choices	_____	3	_____ %
Individual rights & retention	_____	4	_____ %
Vendors, security & risk	_____	6	_____ %
TOTAL	_____	18	_____ %

16-18 Good operational baseline: Still obtain legal review for scope and wording.

12-15 Material gaps remain: Resolve before broad collection of personal data.

6-11 High privacy risk: Minimize data and obtain specialist guidance.

0-5 Not ready: Do not launch personal-data processing without corrective work.

5. MVP Health Audit

A 25-point scorecard across security, performance, UX, code quality and deployment readiness.

25 ITEMS

Use: Pass / Review / Fail / N/A

1. No known critical security vulnerabilities

Dimension: Security

Evidence: Review open critical/high findings and confirm blockers are resolved or formally accepted.

☐ Pass (1) ☐ Fail/Unknown (0) Notes: _____

2. Authentication and session flows work reliably

Dimension: Security

Evidence: Test signup, login, logout, reset, expiry and revocation.

☐ Pass (1) ☐ Fail/Unknown (0) Notes: _____

3. Authorization and tenant isolation are enforced

Dimension: Security

Evidence: Verify every role and cross-user/tenant attempt.

☐ Pass (1) ☐ Fail/Unknown (0) Notes: _____

4. Secrets and production configuration are protected

Dimension: Security

Evidence: Check repository, build output, environment and access permissions.

☐ Pass (1) ☐ Fail/Unknown (0) Notes: _____

5. Security monitoring and patch ownership exist

Dimension: Security

Evidence: Confirm alerts, dependency scanning and an owner for remediation.

☐ Pass (1) ☐ Fail/Unknown (0) Notes: _____

6. Core pages meet response-time targets

Dimension: Performance

Evidence: Measure realistic production-like latency at p50, p95 and p99.

☐ Pass (1) ☐ Fail/Unknown (0) Notes: _____

7. Expected peak load has been tested

Dimension: Performance

Evidence: Run safe load tests against the highest-value journeys.

☐ Pass (1) ☐ Fail/Unknown (0) Notes: _____

8. Database queries and indexes are reviewed

Dimension: Performance

Evidence: Find slow queries, N+1 patterns and missing indexes.

☐ Pass (1) ☐ Fail/Unknown (0) Notes: _____

9. External calls have timeouts and retries

Dimension: Performance

Evidence: Simulate slow/failing dependencies and observe resource use.

☐ Pass (1) ☐ Fail/Unknown (0) Notes: _____

10. Resource and cost limits are defined

Dimension: Performance

Evidence: Review memory, CPU, concurrency, quotas and AI/API spend controls.

☐ Pass (1) ☐ Fail/Unknown (0) Notes: _____

11. A new user can understand the product quickly

Dimension: UX

Evidence: Observe an unfamiliar tester completing the primary task.

☐ Pass (1) ☐ Fail/Unknown (0) Notes: _____

12. Primary journey has no avoidable blockers

Dimension: UX

Evidence: Walk through signup to first value with realistic data.

☐ Pass (1) ☐ Fail/Unknown (0) Notes: _____

13. Errors explain what happened and what to do

Dimension: UX

Evidence: Trigger common validation, network and permission failures.

☐ Pass (1) ☐ Fail/Unknown (0) Notes: _____

14. Mobile and responsive layouts are usable

Dimension: UX

Evidence: Test supported breakpoints, orientation, keyboard and touch targets.

☐ Pass (1) ☐ Fail/Unknown (0) Notes: _____

15. Basic accessibility checks pass

Dimension: UX

Evidence: Test keyboard, focus, labels, contrast, zoom and screen-reader names.

☐ Pass (1) ☐ Fail/Unknown (0) Notes: _____

16. Repository structure and ownership are understandable

Dimension: Code Quality

Evidence: Ask a developer unfamiliar with the app to locate major components and owners.

☐ Pass (1) ☐ Fail/Unknown (0) Notes: _____

17. Automated tests cover critical logic

Dimension: Code Quality

Evidence: Map critical risks to unit, integration and end-to-end tests.

☐ Pass (1) ☐ Fail/Unknown (0) Notes: _____

18. Linting, type checks and builds pass

Dimension: Code Quality

Evidence: Run all quality gates from a clean checkout.

☐ Pass (1) ☐ Fail/Unknown (0) Notes: _____

19. Dependencies are maintained and intentional

Dimension: Code Quality

Evidence: Review outdated, unused, abandoned and duplicate packages.

☐ Pass (1) ☐ Fail/Unknown (0) Notes: _____

20. Operational documentation is sufficient

Dimension: Code Quality

Evidence: Follow setup, deploy and incident instructions without tribal knowledge.

☐ Pass (1) ☐ Fail/Unknown (0) Notes: _____

21. Build and deployment are reproducible

Dimension: Deployment

Evidence: Deploy a tagged commit using only documented automation.

☐ Pass (1) ☐ Fail/Unknown (0) Notes: _____

22. Database migrations are safe

Dimension: Deployment

Evidence: Test migration on production-like data and recovery strategy.

☐ Pass (1) ☐ Fail/Unknown (0) Notes: _____

23. Monitoring, logs and health checks are active

Dimension: Deployment

Evidence: Verify dashboards, alerts, correlation and readiness signals.

☐ Pass (1) ☐ Fail/Unknown (0) Notes: _____

24. Backups and restore have been tested

Dimension: Deployment

Evidence: Restore a recent backup and record recovery time and gaps.

☐ Pass (1) ☐ Fail/Unknown (0) Notes: _____

25. Rollback and incident response are ready

Dimension: Deployment

Evidence: Simulate a failed release and follow the runbook.

☐ Pass (1) ☐ Fail/Unknown (0) Notes: _____

Dimension	Passed	Total	Score
Security	____	5	____ %
Performance	____	5	____ %
UX	____	5	____ %
Code Quality	____	5	____ %
Deployment	____	5	____ %
TOTAL	____	25	____ %

22-25 Healthy MVP: Strong baseline; review any high-impact failed item.

17-21 Promising but exposed: Create a pre-launch improvement sprint.

10-16 Fragile MVP: Major technical or product risks remain.

0-9 Prototype stage: Do not treat the product as production-ready.

6. Launch Checklist

A 20-item final pre-flight review covering domain and DNS, accounts, payments, reliability, legal readiness and the Go / No-Go decision.

1. Production domain is connected correctly

Area: Domain and access

Evidence: Confirm the intended production domain resolves to the correct hosting environment and no temporary preview URL is being used.

☐ Pass ☐ Needs work ☐ N/A Owner: _____ Due: _____

2. HTTPS is active and valid

Area: Domain and access

Evidence: Open the production site over HTTPS, confirm the certificate is valid, and verify HTTP redirects to HTTPS.

☐ Pass ☐ Needs work ☐ N/A Owner: _____ Due: _____

3. Domain redirects are consistent

Area: Domain and access

Evidence: Confirm www/non-www and any legacy domains redirect to one canonical production address without loops.

☐ Pass ☐ Needs work ☐ N/A Owner: _____ Due: _____

4. Important pages and routes work

Area: Domain and access

Evidence: Test the homepage, authentication, dashboard, key deep links, error pages and any public landing pages directly.

☐ Pass ☐ Needs work ☐ N/A Owner: _____ Due: _____

5. Test and demo accounts are removed or restricted

Area: Accounts and permissions

Evidence: Review production users and disable seed, demo, developer and shared test accounts that are not required.

☐ Pass ☐ Needs work ☐ N/A Owner: _____ Due: _____

6. Administrator access is limited

Area: Accounts and permissions

Evidence: List all administrators, confirm each still requires access, and remove unnecessary privileges.

☐ Pass ☐ Needs work ☐ N/A Owner: _____ Due: _____

7. Account lifecycle works end to end

Area: Accounts and permissions

Evidence: Test registration, verification, login, logout, password reset, session expiry and account closure in production.

☐ Pass ☐ Needs work ☐ N/A Owner: _____ Due: _____

8. Roles and permissions have been tested

Area: Accounts and permissions

Evidence: Test each user role against allowed and forbidden actions, including direct API and URL access.

☐ Pass ☐ Needs work ☐ N/A Owner: _____ Due: _____

9. Live payment configuration is enabled

Area: Payments and communication

Evidence: Confirm production payment keys and webhooks are active and sandbox or test-mode indicators are disabled.

☐ Pass ☐ Needs work ☐ N/A Owner: _____ Due: _____

10. A real payment and refund have been tested

Area: Payments and communication

Evidence: Complete a low-value live transaction, verify records and notifications, then process and confirm a refund.

☐ Pass ☐ Needs work ☐ N/A Owner: _____ Due: _____

11. Transactional emails are delivered

Area: Payments and communication

Evidence: Trigger signup, reset, order, invoice or other critical emails and verify delivery, formatting and links.

☐ Pass ☐ Needs work ☐ N/A Owner: _____ Due: _____

12. Sender and reply settings are correct

Area: Payments and communication

Evidence: Confirm sender name, from address, reply-to address, domain authentication and support routing.

☐ Pass ☐ Needs work ☐ N/A Owner: _____ Due: _____

13. Production backups are active

Area: Reliability

Evidence: Verify scheduled backups cover databases, uploaded files and other stateful data, with expected retention.

☐ Pass ☐ Needs work ☐ N/A Owner: _____ Due: _____

14. Restore steps have been tested

Area: Reliability

Evidence: Restore a recent backup in a safe environment and record the result, duration and responsible owner.

☐ Pass ☐ Needs work ☐ N/A Owner: _____ Due: _____

15. Error monitoring and alerts are active

Area: Reliability

Evidence: Trigger a controlled test error and confirm it appears in monitoring and reaches the correct person or channel.

☐ Pass ☐ Needs work ☐ N/A Owner: _____ Due: _____

16. Critical integrations work in production

Area: Reliability

Evidence: Test forms, APIs, webhooks, storage, search, AI providers and other dependencies using production-safe scenarios.

☐ Pass ☐ Needs work ☐ N/A Owner: _____ Due: _____

17. Legal pages and consent controls are published

Area: Legal and customer readiness

Evidence: Verify Privacy Policy, Terms, imprint where applicable, cookie controls and consent records are accessible and current.

☐ Pass ☐ Needs work ☐ N/A Owner: _____ Due: _____

18. Customer support is ready

Area: Legal and customer readiness

Evidence: Confirm support contact details, ownership, expected response time, escalation path and basic help content.

☐ Pass ☐ Needs work ☐ N/A Owner: _____ Due: _____

19. Analytics collect only approved data

Area: Legal and customer readiness

Evidence: Review production analytics and logs for personal data, secrets, sensitive fields and tracking before consent.

☐ Pass ☐ Needs work ☐ N/A Owner: _____ Due: _____

20. Final launch decision is documented

Area: Launch governance

Evidence: Record Go, Conditional Go or No-Go, unresolved risks, accepted exceptions, owners and the approval date.

☐ Pass ☐ Needs work ☐ N/A Owner: _____ Due: _____

Launch result

Passed: ____ / 20 Needs work: ____ N/A: ____

GO: No unresolved launch blockers. CONDITIONAL GO: Known issues have owners, deadlines and accepted risk. NO-GO: A critical security, privacy, payment, data-loss or core-journey issue remains.

Launch Action Plan

Convert every failed or uncertain item into an owned action. Do not use the overall score to hide a single severe risk.

Priority	Finding	Risk/Impact	Owner	Due date	Evidence / Status
☐ Blocker ☐ Before launch ☐ Later					
☐ Blocker ☐ Before launch ☐ Later					
☐ Blocker ☐ Before launch ☐ Later					
☐ Blocker ☐ Before launch ☐ Later					
☐ Blocker ☐ Before launch ☐ Later					
☐ Blocker ☐ Before launch ☐ Later					
☐ Blocker ☐ Before launch					

☐ Later					
☐ Blocker ☐ Before launch ☐ Later					
☐ Blocker ☐ Before launch ☐ Later					
☐ Blocker ☐ Before launch ☐ Later					
☐ Blocker ☐ Before launch ☐ Later					
☐ Blocker ☐ Before launch ☐ Later					

Launch decision

- ☐ GO - launch criteria met
- ☐ CONDITIONAL GO - accepted risks documented
- ☐ NO-GO - blockers remain

Decision owner: _____ Date: _____

Conditions / accepted risks:

Reference foundation

This original Scale200 checklist package was informed by widely used security and privacy frameworks, including the OWASP Application Security Verification Standard, OWASP API Security Top 10, and European Commission guidance on GDPR principles and transparency. It is intentionally simplified for founders and small teams.

- OWASP Application Security Verification Standard (ASVS): owasp.org/www-project-application-security-verification-standard/
- OWASP API Security Top 10 (2023): owasp.org/API-Security/editions/2023/en/0x11-t10/
- European Commission - GDPR principles and business obligations: commission.europa.eu/law/law-topic/data-protection/
- California Attorney General - CCPA information: oag.ca.gov/privacy/ccpa

© 2026 Scale200. You may provide this kit to your own clients or team unchanged. Do not resell, remove attribution, or claim that completion constitutes certification.